C.A.S.A

Context, Architecture, Stack & Automation

Agent-Native Architecture Manual

A complete open method for building, governing and modernizing software systems with AI coding agents

---

Version 0.1 - Public Draft

Prepared for publication as an open methodology

Date: May 26, 2026

> Core principle: Design for agents. Govern for humans. Evolve for legacy.

Table of Contents

Part I - Method Foundation

1. Executive Summary

C.A.S.A stands for Context, Architecture, Stack & Automation. It is an agent-native software architecture method designed for teams that use AI coding agents such as Cursor, Codex, Claude Code, Devin, Antigravity, Copilot-style systems, and future agent platforms.

The method is not a framework, not a single stack, and not a prompt library. C.A.S.A is an operating layer for software delivery. It gives agents the right context, the right boundaries, the right reusable capabilities, and the right automation so that AI-assisted development remains maintainable instead of drifting into chaos.

C.A.S.A supports two adoption modes: Greenfield Mode for new systems and Brownfield Mode for existing or legacy systems. Greenfield Mode bootstraps structure, contracts, policies, specs, documentation and automation from day one. Brownfield Mode overlays C.A.S.A on top of an existing codebase without forcing a rewrite, then adds discovery, characterization tests, seams, strangler migration plans and governance.

> C.A.S.A is not a template. C.A.S.A is an agent-native architecture layer for building, governing and modernizing software systems.

1.1 What C.A.S.A optimizes

- Lower architecture drift when humans and agents modify the same codebase.
- Lower token waste through progressive context, scoped skills and generated adapters.
- Higher reuse through a single C.A.S.A Core that generates tool-specific files.
- Better security through policies, protected paths, risk levels and CI gates.
- Better legacy modernization through discovery-first and strangler-first workflows.
- Better public adoption through open standards such as AGENTS.md, Agent Skills, MCP, A2A and OpenAPI.

1.2 Public definition

C.A.S.A is an open, agent-native software architecture method that organizes context, architecture, stack decisions and automation into a reusable operating layer for AI-assisted software delivery.

2. Why C.A.S.A Exists

AI coding agents are increasingly able to read code, edit files, run commands, write tests and coordinate multi-step tasks. However, without constraints they amplify the same problems that already hurt software teams: duplicated patterns, weak boundaries, hidden coupling, missing tests, inconsistent documentation and security shortcuts.

C.A.S.A treats AI coding as a governance problem, not only a productivity problem. The goal is not just to produce more code. The goal is to produce code that remains understandable, testable, secure, documented and evolvable.

2.1 The main failure modes

- Context dumping: too many instructions are loaded into every task, increasing token cost and reducing signal.
- Tool lock-in: rules are written only for one IDE or one agent and cannot be reused elsewhere.
- Architecture drift: agents replicate accidental patterns and slowly move the codebase away from intended design.

- Spec bypass: implementation starts before requirements, acceptance criteria and risks are clear.
- Security theater: policies exist in text but are not enforced by workflows, CI, sensors or approvals.
- Legacy hostility: methods assume a clean new project and fail when applied to messy existing systems.

2.2 C.A.S.A answer

C.A.S.A separates stable project knowledge from tool-specific adapters. It uses progressive disclosure: a small global map, scoped rules, skills loaded on demand, subagents for isolated exploration, references for long-form knowledge, and deterministic sensors for feedback.

Problem	C.A.S.A answer
Agents need context	AGENTS.md, Context Fabric, repo maps, specs and policies
Agents need capabilities	Skills, subagents, workflows, hooks and playbooks
Tools differ	Adapter generator from one C.A.S.A Core
Architecture drifts	Governance engine, sensors, evals and architecture review
Legacy systems resist change	Brownfield overlay, discovery, tests, seams and strangler migration

3. The Four Pillars

3.1 Context

Context is the project's shared memory for humans and agents. It includes AGENTS.md, repo maps, specs, policies, architecture diagrams, domain maps, dependency maps, decision records and generated tool guidance.

3.2 Architecture

Architecture defines boundaries. It decides where code belongs, which dependencies are allowed, how modules communicate, how contracts are versioned and how legacy systems are modernized. C.A.S.A recommends modular monolith first for many new systems, with deliberate paths to microservices only when the domain and operational needs justify it.

3.3 Stack

Stack is the recommended technology set for the current project type. C.A.S.A separates stack recommendations from the method itself. The reference full-stack blueprint uses TypeScript, Next.js, NestJS, PostgreSQL, OpenAPI, Docker, CI/CD, observability and DevSecOps checks, but the method can be adapted to other ecosystems.

3.4 Automation

Automation makes the method enforceable. It includes tests, type checks, OpenAPI generation, CI/CD gates, security scans, migration checks, documentation generation, adapter generation, drift checks and C.A.S.A doctor validation.

> The four pillars are not independent. Context guides Architecture. Architecture constrains Stack. Stack is protected by Automation. Automation feeds Context back into the system.

Part II - C.A.S.A 2.0 Reference Architecture

4. The Agent-Native Architecture Model

C.A.S.A 2.0 is designed around a core idea: the method must remain stable even as agent products change. Cursor, Codex, Claude Code, Devin and Antigravity may evolve quickly. C.A.S.A should not be coupled to any of them. Instead, it defines a neutral core and generates adapters.

```
C.A.S.A Core
  -> Adapter Generator
      -> Cursor rules
      -> Codex skills
      -> Claude skills/plugins
      -> Devin knowledge/playbooks
      -> Antigravity workflows
      -> Generic AGENTS.md instructions

Greenfield systems use C.A.S.A to create structure.
Brownfield systems use C.A.S.A as an overlay before modernization.
```

4.1 Main layers

Layer	Purpose
Kernel	Defines method primitives, standards, policies, risk model and artifact taxonomy.
Context Fabric	Maps repository, domain, runtime, dependencies, APIs, databases and legacy inventory.
Capability Layer	Holds skills, subagents, workflows, hooks, commands and playbooks.
Protocol Layer	Uses AGENTS.md, Agent Skills, MCP, A2A, OpenAPI, AsyncAPI and related standards.
Governance Engine	Controls permissions, policies, risk levels, sensors, evals and audit trails.
Modernization Layer	Supports legacy discovery, characterization tests, seams, strangler migration and retirement.
Adapter Layer	Generates files for each tool without duplicating source knowledge.
Automation Layer	Runs doctor checks, CI gates, tests, security scans and documentation generation.

5. C.A.S.A Kernel

The Kernel is the source of truth. It contains stable definitions and policy files that should not be written separately for each agent. Tool-specific files are generated from the Kernel.

```
.casa/kernel/
  principles/
    context.md
    architecture.md
    stack.md
    automation.md
  ontology/
    artifact-types.md
    agent-roles.md
    risk-taxonomy.md
  standards/
    frontend.md
    backend.md
    database.md
    api.md
    devops.md
    testing.md
    documentation.md
    legacy.md
  policies/
    secure-coding.md
    api-security.md
    database-security.md
    infrastructure-security.md
    supply-chain-security.md
    data-protection.md
  contracts/
    openapi.md
    asyncapi.md
    events.md
  risk-model/
    risk-levels.md
    approval-matrix.md
```

5.1 Kernel rules

- Edit Kernel files, not generated agent files.
- Use short global rules and longer references only when needed.
- Every standard must be testable, reviewable or enforceable.
- Every policy must identify when it applies and how it is checked.
- Every risky action must map to a risk level and approval policy.

6. Context Fabric

Context Fabric is the living map of the system. It prevents agents from rediscovering the codebase from scratch on every task. In a new project, it starts small. In a legacy project, it becomes the first deliverable.

```
.casa/context/
  repo-map/
    modules.md
    dependency-map.md
    ownership-map.md
    runtime-map.md
  architecture-map/
    c4-context.md
    c4-container.md
    c4-components.md
  domain-map/
    domains.md
    bounded-contexts.md
    business-capabilities.md
  code-intelligence/
    symbols.index.json
    call-graph.json
    dependency-graph.json
    api-surface.json
  legacy-inventory/
    systems.md
    modules.md
    databases.md
    batch-jobs.md
    integrations.md
    risks.md
```

6.1 Minimum viable context

- A repository map showing major folders, apps, packages and ownership.
- A dependency map showing internal packages, external services and risky coupling.
- An API map showing endpoints, contracts and generated clients.
- A database map showing schemas, tables, migrations and data ownership.
- A runtime map showing deployables, environments, queues, jobs and scheduled tasks.
- A domain map showing capabilities, bounded contexts and terminology.

For large codebases, C.A.S.A should eventually integrate AST, LSP, call graph and search/indexing tools. The goal is to give agents structured understanding, not just tokenized text.

7. Protocol Layer

C.A.S.A is protocol-first. It should prefer open or broadly adopted formats over tool-specific conventions whenever possible.

Protocol or format	C.A.S.A role
AGENTS.md	Universal project instructions and repository map for agents.
Agent Skills	Reusable task-specific capabilities with instructions, references and scripts.
MCP	Tool and data access layer for agents.
A2A	Agent-to-agent interoperability pattern for future multi-agent ecosystems.
OpenAPI	HTTP API contracts, Swagger docs, generated clients and contract tests.
AsyncAPI	Event, queue and message contracts.
ADR	Architecture decision records with context and consequences.
C4	Architecture diagrams at context, container and component levels.

7.1 Why protocol-first matters

- It reduces vendor lock-in.
- It makes the method reusable across Cursor, Codex, Claude, Devin, Antigravity and future tools.
- It allows public open-source distribution without depending on a proprietary IDE.

- It makes context easier to validate, version and audit.

8. Agent Capability Layer

The Capability Layer is where C.A.S.A turns knowledge into reusable behavior. It separates skills, subagents, workflows and hooks.

Artifact	Meaning	Example
Skill	Reusable knowledge and procedure loaded on demand.	frontend-design-engineer
Subagent	Specialized worker with isolated context and tool permissions.	legacy-archaeologist
Workflow	Ordered process for a task type.	implement-feature.workflow.md
Hook	Deterministic action before or after agent lifecycle events.	before-migration-guard
Command	Shortcut that triggers a workflow.	/casa-review
Playbook	Human-readable runbook for repeated work.	modernize-module.md

8.1 Recommended core skills

- casa-skill-router
- frontend-design-engineer
- anti-ai-ui-reviewer
- backend-clean-architect
- api-contract-engineer
- database-architect
- security-reviewer
- devops-engineer
- test-harness-engineer
- documentation-writer
- legacy-archaeologist
- migration-planner
- architecture-reviewer
- performance-reviewer

8.2 Recommended subagents

- codebase-cartographer: maps code structure, dependencies and hotspots.
- legacy-archaeologist: explains old modules before anyone changes them.
- domain-extractor: identifies business capabilities and vocabulary.
- security-reviewer: searches for auth, data, secrets and exposure risks.
- test-reviewer: checks whether behavior is protected by tests.
- migration-planner: proposes incremental modernization steps.
- api-contract-reviewer: validates OpenAPI and generated client impact.
- devops-reviewer: checks CI/CD, infrastructure and environment effects.

Rule: load capability, not clutter. A task should activate the smallest useful set of skills and subagents.

9. Governance Engine

The Governance Engine makes C.A.S.A enforceable. It controls what agents may do, which changes require human approval, what sensors must run and how decisions are audited.

```
.casa/governance/
  permissions/
    agent-permissions.md
    protected-files.md
    dangerous-actions.md
  policies/
    secure-coding.md
    api-security.md
    database-security.md
    infra-security.md
    supply-chain-security.md
    data-protection.md
  risk-model/
    risk-levels.md
    approval-matrix.md
    rollback-policy.md
  evals/
    architecture-drift.eval.md
    security-regression.eval.md
    api-contract.eval.md
    ui-quality.eval.md
  sensors/
    lint.sensor.md
    typecheck.sensor.md
    test.sensor.md
    openapi-diff.sensor.md
    migration-risk.sensor.md
    dependency-risk.sensor.md
  audit/
    agent-actions.log.md
    decisions.log.md
    generated-files.log.md
```

9.1 Risk levels

Risk	Examples	Required control
Low	Copy changes, small UI state, documentation update	Agent can proceed with normal tests.
Medium	New endpoint, non-destructive migration, new dependency	Specialist review, tests and contract update.
High	Auth, permissions, secrets, prod infra, destructive migration	Human approval, rollback plan, security review and CI gates.
Critical	Production data deletion, credential rotation, public exposure change	Manual change window, incident-style checklist and two-person review.

9.2 Protected paths

```
protected_paths:
  - devops/terraform/**
  - .github/workflows/**
  - apps/api/src/auth/**
  - apps/api/src/security/**
  - apps/api/prisma/migrations/**
  - packages/contracts/openapi/**
  - .casa/kernel/policies/**
```

9.3 Agent permission policy

- Agents may read repository files and edit only files related to the assigned task.
- Agents must not delete files, disable checks or change secrets without explicit approval.
- Agents must not edit generated adapter files directly; they must edit C.A.S.A Core and regenerate.
- Agents must run required sensors before proposing completion.

- Agents must update specs, contracts, docs and tests when behavior changes.

10. Orchestration Layer

Orchestration decides how agents work together. C.A.S.A uses routers, teams, handoffs and approvals to make multi-agent work controlled instead of chaotic.

```
.casa/orchestration/
  router/
    skill-router.md
    subagent-router.md
    risk-router.md
  teams/
    feature-team.team.md
    legacy-modernization-team.team.md
    security-review-team.team.md
  handoffs/
    planner-to-implementer.md
    implementer-to-reviewer.md
    reviewer-to-fixer.md
  task-ledger/
    active/
    done/
    blocked/
  approvals/
    human-approval-required.md
    auto-approval-rules.md
```

10.1 Team patterns

Team	Members
Feature Team	planner, frontend-engineer, backend-engineer, api-contract-engineer, test-engineer, reviewer
Legacy Team	legacy-archaeologist, dependency-mapper, domain-extractor, characterization-test-writer, migration-planner
Security Team	threat-modeler, security-reviewer, database-security-reviewer, infra-security-reviewer, supply-chain-reviewer
Design Team	frontend-design-engineer, anti-ai-ui-reviewer, accessibility-reviewer, performance-reviewer

10.2 Routing principle

- Low-risk tasks: a single agent may execute with scoped skills.
- Medium-risk tasks: one implementer plus one reviewer or sensor loop.
- High-risk tasks: orchestrator, specialist reviewer, human approval and rollback plan.
- Exploration-heavy tasks: spawn subagents to research independently, then consolidate summaries.

Part III - Greenfield and Brownfield Adoption

11. Greenfield Mode

Greenfield Mode is for new projects. It creates a clean architecture from day one and gives agents the structure they need to avoid inventing patterns. The recommended default for many products is a modular monolith before microservices. Microservices can emerge later when business boundaries, team topology and operational needs become clear.

11.1 Greenfield bootstrap command

```
casa init --mode greenfield --stack next-nest-postgres
casa generate adapters --agents cursor,codex,claude,devin,antigravity
casa doctor
```

11.2 Greenfield output

```
project-root/
  AGENTS.md
  casa.manifest.yaml
  .casa/
  apps/
    web/
    api/
  packages/
    ui/
    contracts/
    api-client/
    config/
    testing/
  specs/
  policies/
  docs/
  devops/
```

11.3 Greenfield first ten decisions

- Choose project mode and target architecture.
- Define the main business capabilities and first bounded contexts.
- Create AGENTS.md as a short map, not a manifesto.
- Define protected paths and agent permissions.
- Create initial specs template and acceptance criteria template.
- Create OpenAPI contract strategy and generated client location.
- Create database migration policy.
- Create security policies and risk levels.
- Create CI quality gates and security checks.
- Generate adapters for the agents the team actually uses.

12. Brownfield Mode for Legacy Systems

Brownfield Mode is for existing projects. It must not start by reorganizing folders or forcing Clean Architecture. It starts as an overlay. The first goal is understanding and safety, not refactoring.

12.1 Brownfield bootstrap command

```
casa init --mode brownfield
casa legacy scan --read-only
casa context build
casa doctor
```

12.2 Brownfield output

```
legacy-project/
  AGENTS.md
  casa.manifest.yaml
  .casa/
    context/
      repo-map/
      legacy-inventory/
      dependency-map/
      hotspots/
    modernization/
      discovery/
      baselining/
      seams/
      strangler/
      branch-by-abstraction/
      anti-corruption/
      retirement/
    capabilities/
      skills/
        legacy-archaeologist/
        characterization-test-writer/
        migration-planner/
        dependency-mapper/
    governance/
      permissions/
      risk-model/
      sensors/
  specs/modernization/
  docs/architecture/
  docs/adr/
```

12.3 Brownfield phases

Phase	Goal	Outputs
Discovery	Understand before changing.	Inventory, repo map, hotspots, integrations, data map.
Safety Baseline	Protect current behavior.	Characterization tests, smoke tests, contract tests, golden masters.
Encapsulation	Create seams and stable boundaries.	Facade, adapter, anti-corruption layer, branch by abstraction plan.
Strangler Migration	Move behavior incrementally.	Routing plan, toggles, new module, parallel run, metrics.
Retirement	Remove legacy safely.	Decommission plan, rollback plan, archive and retention decisions.

Legacy rule: map first, test second, encapsulate third, migrate fourth, delete last.

12.4 Legacy modernization patterns

- Strangler Fig: gradually route behavior from old implementation to new implementation.
- Branch by Abstraction: introduce an abstraction layer so old and new implementations can coexist.
- Anti-Corruption Layer: translate between legacy models and new domain models.
- Facade: create a stable entry point for unstable or messy internals.
- Characterization Testing: capture current behavior before refactoring.
- Parallel Run: compare old and new outputs before switching traffic.
- Feature Toggles: enable gradual rollout and rollback.

13. Reference Repository Structure

The reference structure below is intended for the C.A.S.A open starter. Existing codebases should not be forced into it immediately. New systems can adopt it directly.

```
project-root/
  AGENTS.md
  casa.manifest.yaml

  .casa/
    kernel/
    context/
    capabilities/
    orchestration/
    governance/
    specs/
    modernization/
    protocols/
    adapters/
    generated/
    telemetry/

  apps/
    web/
    api/
    docs/

  packages/
    ui/
    contracts/
    api-client/
    config/
    domain-shared/
    testing/
    observability/

  specs/
    0001-feature-name/
      spec.md
      plan.md
      tasks.md
      acceptance-criteria.md
      threat-model.md

  policies/
    security/
    governance/
    architecture/

  docs/
    adr/
    architecture/
    standards/
    runbooks/

  devops/
    docker/
    compose/
    terraform/
    k8s/
    observability/
    security/
    scripts/
```

13.1 Generated files

Generated files are disposable. They are specific translations of the same C.A.S.A Core into the conventions of each agent platform.

```
Do not edit generated files directly.
Edit .casa/kernel or .casa/capabilities and regenerate adapters.

Generated examples:
  .cursor/rules/*.mdc
  .agents/skills/*/SKILL.md
  .claude/skills/*/SKILL.md
  .devin/repo-knowledge/*.md
  .antigravity/workflows/*.md
```

14. Agent Compatibility Layer

The Agent Compatibility Layer prevents duplication. C.A.S.A should maintain one neutral source of truth and generate files for each agent.

Tool	Generated artifacts
Universal	AGENTS.md, casa.manifest.yaml, specs, policies, docs
Cursor	.cursor/rules/*.mdc, project rules with globs
Codex	.agents/skills/*/SKILL.md, repo-scoped skills, AGENTS.md guidance
Claude Code	.claude/skills, plugins, hooks, commands and subagents
Devin	.devin/instructions.md, repo knowledge, task playbooks
Antigravity	.antigravity/rules, workflows and agent maps
Generic	Plain Markdown instructions and playbooks for any agent

14.1 Adapter generator commands

```
casa adapter add cursor codex claude devin antigravity
casa generate adapters
casa doctor --check-generated
```

14.2 Adapter source model

```
.casa/capabilities/skills/frontend-design-engineer.md
  -> .cursor/rules/frontend-design-engineer.mdc
  -> .agents/skills/frontend-design-engineer/SKILL.md
  -> .claude/skills/frontend-design-engineer/SKILL.md
  -> .devin/repo-knowledge/frontend-design-engineer.md
  -> .antigravity/workflows/frontend-design-engineer.md
```

Part IV - Stack, Contracts and Automation

15. Reference Stack

C.A.S.A is stack-adaptable. The following stack is the reference blueprint for a modern TypeScript product using web, API, database, DevOps and AI agents.

Layer	Reference choice
Monorepo	pnpm workspaces, Turborepo
Frontend	Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, Radix UI, Motion
Frontend data	TanStack Query, TanStack Table, React Hook Form, Zod, Recharts
Backend	NestJS, Fastify adapter, OpenAPI/Swagger, Prisma
Database	PostgreSQL, Redis, S3-compatible storage, optional pgvector
Contracts	OpenAPI, generated TypeScript client, contract tests
Docs	Swagger UI, Docusaurus, Storybook, ADR, C4
DevOps	Docker, Docker Compose, GitHub Actions or GitLab CI, Terraform, OpenTelemetry
Security	OWASP ASVS, NIST SSDF, SLSA, Trivy, CodeQL, secret scanning

15.1 Stack selection rules

- Default to boring, well-documented tools unless the problem requires specialization.
- Prefer ecosystems with strong examples because agents perform better with common patterns.
- Never add a new production dependency without a reason, alternative analysis and maintenance impact.
- Prefer contracts and generated clients over manually duplicated types.
- Prefer deterministic tools for validation: typecheck, lint, tests, OpenAPI diff, migration checks.

16. Frontend Architecture

C.A.S.A frontend architecture is feature-first plus design system. Shared UI primitives live in packages/ui or components/ui. Product-specific components live inside features.

```
apps/web/src/
  app/
    (public)/
    (app)/
  components/
    ui/
    layout/
    feedback/
  features/
    users/
      components/
      hooks/
      schemas/
      services/
      types/
    billing/
    dashboard/
  lib/
    api/
    auth/
    config/
    formatters/
  styles/
    globals.css
    tokens.css
```

16.1 Frontend rules for agents

- components/ui must remain reusable and product-agnostic.
- features/* contains product behavior, screens, hooks and feature-specific components.
- Use design tokens before inventing ad-hoc utility classes.
- Include loading, empty and error states for data-driven UI.
- Do not use generic AI-looking UI patterns: meaningless icons, repeated card grids, vague copy or excessive gradients.
- Use Storybook for component states and edge cases when components become part of the design system.

17. Backend Architecture

The backend reference architecture is modular and Clean Architecture inspired. Each module separates domain, application, infrastructure and presentation.

```
apps/api/src/modules/users/
  domain/
    entities/
    value-objects/
    events/
    repositories/
  application/
    use-cases/
    dtos/
    ports/
  infrastructure/
    prisma/
    mappers/
    repositories/
    services/
  presentation/
    controllers/
    requests/
    responses/
```

17.1 Dependency rule

- Domain imports no framework, ORM or HTTP concepts.
- Application orchestrates use cases and ports but does not know controllers.
- Infrastructure implements repositories, adapters and external gateways.

- Presentation handles controllers, request DTOs, response DTOs and Swagger decorators.
- Prisma and database details must not leak into domain or use cases.
- Controllers must not contain business rules.

17.2 Common patterns

Pattern	Use
Repository	Hide persistence details and protect domain from ORM leakage.
Adapter	Integrate external services behind ports.
Use Case / Command	Represent one application action.
Mapper	Translate API, domain and persistence models.
Domain Event	Decouple side effects from core domain actions.
Outbox	Reliable event publishing when data and event must stay consistent.
Specification	Composable business filters or policies.
Unit of Work	Coordinate transaction boundaries when needed.

18. API Contracts and Documentation

APIs are contracts, not just controller methods. C.A.S.A requires OpenAPI for HTTP APIs and generated clients for frontend consumption when possible.

```
apps/api
  -> generates OpenAPI JSON
packages/contracts
  -> stores versioned openapi.json
packages/api-client
  -> generates TypeScript client
apps/web
  -> consumes API through typed hooks
```

18.1 API policy

- All API routes must be versioned, for example /api/v1.
- Every endpoint must declare auth status: public, authenticated or privileged.
- Request DTOs, response DTOs and domain entities must be separate.
- List endpoints must define pagination and filters.
- Errors must use a standard error envelope.
- Swagger/OpenAPI must be updated when behavior changes.
- Generated clients must be regenerated after contract changes.

18.2 Standard error model

```
{
  "error": {
    "code": "INVOICE_NOT_FOUND",
    "message": "Invoice not found.",
    "requestId": "req_123",
    "details": []
  }
}
```

19. Database and Data Architecture

C.A.S.A treats database changes as high-impact architecture changes. Migrations are code, not operational afterthoughts.

19.1 Data rules

- Every schema change must include migration review.
- Destructive migrations require explicit approval and rollback plan.
- Runtime database credentials should have least privilege.
- Separate migration credentials from runtime credentials when possible.
- Multi-tenant data must be scoped in repositories and tested.
- Sensitive data must be classified and never logged in raw form.
- Backups must be tested, not only configured.
- Production data must not be copied to local environments without masking or anonymization.

19.2 Database security reviewer triggers

- Prisma schema changes.
- New table, column, index or relation.
- Raw SQL or stored procedure changes.
- Tenant isolation or permission-sensitive query changes.
- PII, audit, retention or backup changes.
- Large data migration or destructive migration.

20. DevOps, Observability and Infrastructure

The devops folder is not a dumping ground. It is the versioned operational layer of the system.

```
devops/
  docker/
    api.Dockerfile
    web.Dockerfile
  compose/
    compose.local.yml
    compose.observability.yml
  github-actions/
    ci.yml
    deploy.yml
    security.yml
  terraform/
    environments/
      dev/
      staging/
      prod/
    modules/
      network/
      database/
      app/
  k8s/
    base/
    overlays/
      dev/
      prod/
  observability/
    grafana/
    prometheus/
    otel-collector/
  security/
    trivy/
    codeql/
  scripts/
    migrate.sh
    seed.sh
    backup-db.sh
```

20.1 DevOps policies

- Infrastructure changes must be code-reviewed and versioned.
- Secrets must never be committed to Git.
- Production infrastructure paths are protected paths.
- Deployments must have rollback instructions.
- Observability must include traces, metrics, logs and request correlation IDs.
- Critical services must have health checks and alerting.

21. Security and Governance Policies

C.A.S.A security is layered: policy files define expectations, rules guide agents, skills perform reviews, CI enforces checks and specs capture feature-level risks.

```
policies/security/
  secure-coding.md
  authentication.md
  authorization.md
  api-security.md
  database-security.md
  secrets.md
  infrastructure-security.md
  supply-chain-security.md
  logging-monitoring.md
  incident-response.md
  data-protection.md
```

21.1 Required security controls

- Validate all external input on the server.
- Never trust client-side authorization.
- Never log secrets, tokens, passwords, cookies or sensitive data.
- Do not expose stack traces to API consumers.
- Use least privilege for services, databases and CI permissions.
- Use dependency scanning, secret scanning and container scanning.
- Use threat models for auth, payments, uploads, tenant data and admin features.
- Treat public API, auth, secrets, infra and database migrations as high-risk by default.

21.2 Threat model template

```
# Threat Model

## Assets
- User account
- Session token
- Customer data
- Audit logs

## Trust boundaries
- Browser -> API
- API -> Database
- API -> External provider
- CI/CD -> Production

## Threats
- Broken object-level authorization
- Tenant data leakage
- Token leakage in logs
- Injection through filters
- Unsafe file handling

## Controls
- Backend authorization policy
- Tenant-scoped repository queries
- Request validation
- Audit logging
- Rate limiting
- Error sanitization
```

22. Testing and Quality Gates

C.A.S.A assumes that agents must have feedback sensors. A sensor is a deterministic or semi-deterministic check the agent can run before claiming the work is complete.

22.1 Standard sensors

Sensor	Purpose
lint	Catch style and static code issues.
typecheck	Catch invalid types and broken contracts.
unit tests	Verify isolated behavior.
integration tests	Verify module boundaries and database behavior.
contract tests	Verify API and client compatibility.
e2e tests	Verify user flows.
OpenAPI diff	Detect API breaking changes.
migration risk	Detect destructive or high-risk schema changes.
security scan	Detect dependency, secret, IaC and container risks.
architecture drift	Detect dependency rule violations and unwanted coupling.

22.2 Test expectations

- Every behavior change needs tests or an explicit reason why not.
- Every API change needs contract validation.
- Every migration needs migration safety review.
- Every sensitive authorization path needs negative tests.
- Every legacy modernization step needs characterization or smoke tests before refactoring.

Part V - Workflows, Specs and Templates

23. Specs as the Source of Feature Truth

Specs do not live inside skills. Skills are reusable. Specs are feature-specific. C.A.S.A keeps specs in a dedicated folder and makes them the source of truth for implementation.

```
specs/
  0001-invoices-dashboard/
    spec.md
    plan.md
    tasks.md
    acceptance-criteria.md
    api-contract.md
    data-model.md
    ui-flow.md
    threat-model.md
    questions.md
```

23.1 Spec template

```
# Spec: <Feature Name>

## Goal
What problem will this feature solve?

## Users
Who uses it and what permissions do they have?

## Functional requirements
- Requirement 1
- Requirement 2

## Non-functional requirements
- Performance
- Security
- Accessibility
- Observability

## API requirements
- GET /api/v1/example
- POST /api/v1/example

## Data model
- Entity
- Fields
- Relationships

## Acceptance criteria
- Given <context>, when <action>, then <expected result>.

## Out of scope
- Things not included in this feature.

## Risks and questions
- Unknowns to resolve before implementation.
```

23.2 Spec workflow

- Create spec before coding.
- Add acceptance criteria before implementation.
- Use the skill router to choose skills and subagents.
- Implement only what the spec allows.
- Record ambiguities in questions.md instead of guessing silently.
- Update spec, contracts, tests and docs when scope changes.

24. Skill Router

The Skill Router selects the smallest set of capabilities needed for a task. It prevents loading every skill into every task.

```
# casa-skill-router

Purpose:
Choose the smallest set of C.A.S.A skills and subagents required for a task.

Rules:
- Do not load frontend skills for backend-only tasks.
- Do not load devops skills unless infra, CI/CD or environment files are involved.
- Load security-reviewer when auth, permissions, secrets, public endpoints or sensitive data are
  touched.
- Load database-security-reviewer when schema, migrations, raw SQL or tenant isolation changes.
- Load legacy-archaeologist before changing poorly understood legacy modules.
- Return selected capabilities and why.
```

24.1 Skill routing examples

Task	Skills
Create landing page	frontend-design-engineer, anti-ai-ui-reviewer
Create dashboard	frontend-design-engineer, api-contract-engineer, test-engineer
Create endpoint	backend-clean-architect, api-contract-engineer, test-engineer
Change database schema	database-architect, database-security-reviewer, test-engineer
Change auth	security-reviewer, backend-clean-architect, test-engineer
Change infrastructure	devops-engineer, infra-security-reviewer
Modernize legacy module	legacy-archaeologist, characterization-test-writer, migration-planner

25. Core Skill Template

```
---
name: <skill-name>
description: Use when <clear trigger>. Do not use when <clear boundary>.
---

# Purpose
Explain what this skill helps the agent do.

# Inputs
- Relevant specs
- Relevant policies
- Existing examples
- Constraints

# Workflow
1. Understand the objective.
2. Check applicable policies.
3. Inspect existing examples.
4. Plan minimal changes.
5. Implement.
6. Run required sensors.
7. Review output.
8. Report risks and next steps.

# Quality bar
- Requirement 1
- Requirement 2

# Anti-patterns
- Thing to avoid
- Thing to avoid
```

25.1 Backend clean architect skill example

```
---
name: backend-clean-architect
description: Use when creating or changing backend modules, use cases, repositories,
  controllers, DTOs, services or domain logic.
---

Rules:
- Domain imports no NestJS, Prisma, HTTP or framework code.
- Use cases do not know controllers.
- Controllers do not contain business rules.
- Repositories are ports in domain/application and implementations in infrastructure.
- Request DTOs, response DTOs and domain objects are separate.
- Update OpenAPI and tests when API behavior changes.
```

26. Subagent Template

```
---
name: legacy-archaeologist
description: Explore legacy code before modification. Summarize behavior, dependencies, risks
  and safe seams.
tools: read, search, shell-readonly
risk: low-readonly
---

Mission:
Understand the target legacy area without editing files.

Output:
1. What the module does.
2. Entry points and dependencies.
3. Data touched.
4. External integrations.
5. Hidden risks.
6. Tests that already exist.
7. Candidate seams for safe modernization.
8. Questions for humans.
```

26.1 Subagent usage rules

- Use subagents for exploration-heavy, parallel or noisy tasks.
- Keep subagent context isolated and ask for concise summaries.
- Do not let exploratory subagents modify code unless explicitly allowed.
- Use one subagent per concern when reviewing security, tests, performance or architecture.
- Consolidate subagent output into the main task ledger.

27. Workflow Templates

27.1 Feature implementation workflow

```
1. Read AGENTS.md and relevant spec.
2. Use skill router.
3. Identify risk level and protected paths.
4. Inspect existing examples.
5. Propose affected files.
6. Implement smallest coherent change.
7. Update API contracts if needed.
8. Update tests and docs.
9. Run sensors.
10. Review architecture, security and quality.
11. Summarize completed work and remaining risks.
```

27.2 Legacy modernization workflow

```
1. Run read-only discovery.
2. Build repo and dependency maps.
3. Identify business capability and ownership.
4. Add characterization tests.
5. Create seams or facade.
6. Introduce new implementation behind abstraction.
7. Run old and new in parallel when possible.
8. Move traffic gradually.
9. Verify metrics and tests.
10. Retire old implementation with rollback plan.
```

27.3 Security-sensitive workflow

```
1. Create or update threat-model.md.
2. Activate security-reviewer.
3. Identify assets, trust boundaries and abuse cases.
4. Add authorization and negative tests.
5. Check logging and error exposure.
6. Run dependency, secret, IaC and container scans.
7. Require human approval for high-risk changes.
8. Update runbooks if operational behavior changes.
```

28. C.A.S.A Doctor

casa doctor is the validation command. It should verify whether the repository is following the method and whether generated adapter files are in sync.

```
casa doctor
casa doctor --mode greenfield
casa doctor --mode brownfield
casa doctor --check-generated
casa doctor --security
casa doctor --legacy
```

28.1 Doctor checks

- AGENTS.md exists and is short enough to be useful.
- casa.manifest.yaml is valid.
- Skills have clear names and descriptions.
- Generated adapter files are up to date.
- Specs include acceptance criteria.
- Security policies exist.
- Protected paths are defined.
- OpenAPI exists if API mode is enabled.
- CI runs lint, typecheck, tests and security checks.
- Brownfield projects have a legacy inventory before modernization work.

29. CI/CD Security Gate Example

```yaml
name: Security

on:
  pull_request:
  push:
    branches: [main]

jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
        with:
          version: 10
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: pnpm
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm typecheck
      - run: pnpm test
      - run: pnpm api:openapi:check
      - run: pnpm casa:doctor
      - run: trivy fs --exit-code 1 --severity HIGH,CRITICAL .
```

Part VI - Open-Source Distribution and Roadmap

30. Public Repository Strategy

C.A.S.A should be published as a method, starter kit, CLI, skills pack and legacy modernization kit. A single idea document is not enough. Adoption requires a repository that people can clone, inspect and run.

30.1 Recommended public ecosystem

Repository	Purpose
casa-method/casa	Core method, docs, manifest, policies and specs.
casa-method/casa-cli	init, generate, doctor, legacy scan and adapter commands.
casa-method/casa-agent-adapters	Cursor, Codex, Claude, Devin, Antigravity and generic adapters.
casa-method/casa-skills	Reusable skills and subagents.
casa-method/casa-legacy-kit	Brownfield discovery, characterization tests and modernization workflows.
casa-method/casa-starter	Greenfield starter templates.
casa-method/casa-devsecops	CI, security scans, IaC and observability templates.

30.2 Platform roles

- GitHub: source of truth, issues, discussions, releases, actions and public community.
- GitLab: mirror, alternate CI/CD examples and DevSecOps templates.
- Hugging Face: skills datasets, prompts, demo spaces and model/dataset cards for AI-native distribution.
- NPM: CLI distribution, for example npx @casa-method/cli init.
- Docs site: Docusaurus-based documentation for onboarding and versioned manuals.

30.3 Open-source files

```
README.md
LICENSE
CONTRIBUTING.md
CODE_OF_CONDUCT.md
SECURITY.md
GOVERNANCE.md
CHANGELOG.md
ROADMAP.md
CITATION.cff
```

30.4 License recommendation

- Code, CLI, templates and adapters: Apache-2.0 or MIT.
- Text, methodology and manual: Creative Commons Attribution 4.0.
- Use permissive licensing first if corporate adoption is a goal.
- Make security reporting explicit in SECURITY.md.

31. Roadmap

Version	Scope
v0.1	Manifesto, AGENTS.md, manifest file, 8 core skills, 6 subagents, specs, policies and initial adapters.

Version	Scope
v0.2	Adapter generator, casa doctor, Cursor/Codex/Claude support, generated file sync checks.
v0.3	Brownfield mode, legacy scan, context maps, characterization test templates.
v0.4	C.A.S.A Legacy Kit, strangler workflows, branch by abstraction templates and migration plans.
v0.5	MCP server, code intelligence index, Sourcegraph-compatible context export and drift checks.
v1.0	Stable greenfield and brownfield modes, public registry of skills, docs site and governance model.

31.1 Non-goals

- C.A.S.A is not trying to replace frameworks such as Next.js, NestJS, Rails, Laravel or Spring.
- C.A.S.A is not trying to pick a single AI vendor winner.
- C.A.S.A is not a promise of total security.
- C.A.S.A is not a rewrite-first modernization method.
- C.A.S.A is not a license to let agents modify production-sensitive systems without review.

32. Glossary

Term	Definition
Agent-native	Designed for AI coding agents as first-class users, not as an afterthought.
Adapter	Generated tool-specific representation of C.A.S.A Core guidance.
Brownfield	Existing or legacy project adoption mode.
Capability	Reusable skill, subagent, workflow, hook, command or playbook.
Context Fabric	Structured map of repository, domain, architecture, runtime and legacy knowledge.
Doctor	Validation command that checks C.A.S.A health and compliance.
Greenfield	New project adoption mode.
Kernel	Neutral source of truth for method definitions, standards and policies.
Sensor	Deterministic check an agent can run for feedback.
Skill	Reusable task-specific instructions with optional references and scripts.
Subagent	Specialized worker agent with isolated context and permissions.
Workflow	Ordered task process that agents and humans can follow.

33. Reference Sources

The C.A.S.A manual is an original architecture proposal. The following sources support the open standards, agent workflows, security controls and modernization patterns referenced by the method.

[1] AGENTS.md official site - https://agents.md/

[2] OpenAI Codex - Custom instructions with AGENTS.md - https://developers.openai.com/codex/guides/agents-md

[3] OpenAI Codex - Agent Skills - https://developers.openai.com/codex/skills

[4] OpenAI Codex - Subagents - https://developers.openai.com/codex/subagents

[5] Cursor Docs - Rules - https://cursor.com/docs/rules

[6] Claude Code Docs - Plugins reference - https://code.claude.com/docs/en/plugins-reference

[7] Claude Code Docs - Hooks guide - https://code.claude.com/docs/en/hooks-guide

[8] Devin Docs - Knowledge - https://docs.devin.ai/product-guides/knowledge

[9] Devin Docs - DeepWiki - https://docs.devin.ai/work-with-devin/deepwiki

[10] Model Context Protocol documentation - https://modelcontextprotocol.io/docs/getting-started/intro

[11] Agent2Agent Protocol documentation - https://a2a-protocol.org/latest/

[12] GitHub Spec Kit documentation - https://github.github.com/spec-kit/

[13] OpenAPI Specification - https://swagger.io/specification/

[14] OWASP ASVS - https://owasp.org/www-project-application-security-verification-standard/

[15] NIST Secure Software Development Framework SP 800-218 - https://csrc.nist.gov/pubs/sp/800/218/final

[16] SLSA supply-chain security framework - https://slsa.dev/

[17] Clean Architecture - The Dependency Rule - https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

[18] Martin Fowler - Monolith First - https://martinfowler.com/bliki/MonolithFirst.html

[19] Martin Fowler - Original Strangler Fig Application - https://martinfowler.com/bliki/OriginalStranglerFigApplication.html

[20] Martin Fowler - Branch by Abstraction - https://martinfowler.com/bliki/BranchByAbstraction.html

[21] Thoughtworks Technology Radar - Feedback sensors for coding agents - https://www.thoughtworks.com/en-br/radar/techniques/feedback-sensors-for-coding-agents

[22] Thoughtworks Technology Radar - Codebase cognitive debt - https://www.thoughtworks.com/en-br/radar/techniques/codebase-cognitive-debt

[23] Sourcegraph - Intelligence layer for AI coding agents - https://sourcegraph.com/blog/a-new-era-for-sourcegraph-the-intelligence-layer-for-ai-coding-agents-and-developers

[24] Storybook documentation - https://storybook.js.org/docs

[25] Docusaurus documentation - https://docusaurus.io/

[26] OpenTelemetry documentation - https://opentelemetry.io/docs/

[27] Hugging Face Hub - Model Cards - https://huggingface.co/docs/hub/en/model-cards

Final principle: Write once. Adapt everywhere. Govern always.